Groundwork Evaluation: Spec-Driven vs. Ad-Hoc AI Coding

2 Oct 2026 · @Hemanth

Executive summary

With Groundwork, the same model built a tested, documented app with every decision traceable. It cost $3.31 instead of $0.58 and only 7 more minutes of wall-clock time. Without Groundwork, it built a working demo in 2.5 minutes of compute, with no tests, no docs and a security exposure nobody chose.

What leadership should take from this test:

- **More delivered for the money.** The Groundwork build shipped 36 passing automated tests, 15 traced requirements, 4 themes, full page setup, math and diagrams. The ad-hoc build shipped 0 tests and about a quarter of the features.
- **Decisions are made by people, on the record.** Groundwork stopped the agent at a security tension (scripts in the PDF renderer) and made the owner choose. The choice, its risk and the signer are written down. The ad-hoc build made the opposite trade-offs silently, including listening on every network interface.
- **Work can be handed over.** The Groundwork repo has 11 planning and foundation documents a new engineer or agent can read in 15 minutes. The ad-hoc repo has 3 source files and nothing else.
- **The cost premium is real but small in context.** $2.73 extra and 8.7 extra minutes of model compute. One avoided production bug, review cycle or onboarding day pays for this many times over.
- **It is not magic.** The Groundwork PDF still had layout defects (clipped code lines, orphaned headings). Its own checklist flagged the human visual check as not yet done. The process made the gap visible; a person still has to close it.

Recommendation: pilot Groundwork on one real production feature team for one quarter, measuring escaped defects, review time and onboarding time against a comparable team.

Experiment setup

Two Claude Code sessions built the same product, a local Markdown-to-PDF web app, from the same one-sentence prompt. The only variable was whether the Groundwork plugin was enabled.

> "create a web application which can take markdown file or content as input and then convert into pdf, where user can download the pdf"

The prompt is deliberately thin, like most real feature requests. Without Groundwork, the agent filled every gap with its own guesses. With Groundwork, the agent interviewed the owner first: 19 recorded questions and answers turned the sentence into 11 functional requirements, 4 non-functional ones and 12 acceptance criteria before any code was written.

	Without Groundwork	With Groundwork
Model	Claude Sonnet 5.5	Claude Sonnet 5.5
Plugin	None	groundwork-specflow 0.1.4 (standard 0.5.0)
Path to code	Prompt → code	Interview → RFC → approval → spec → approval → plan → tasks → evals → code
Human gates	None	2 sign-offs (RFC at 04:28, spec at 04:35 UTC)
Repo folder	`without_groundwork/`	`with_groundwork/`

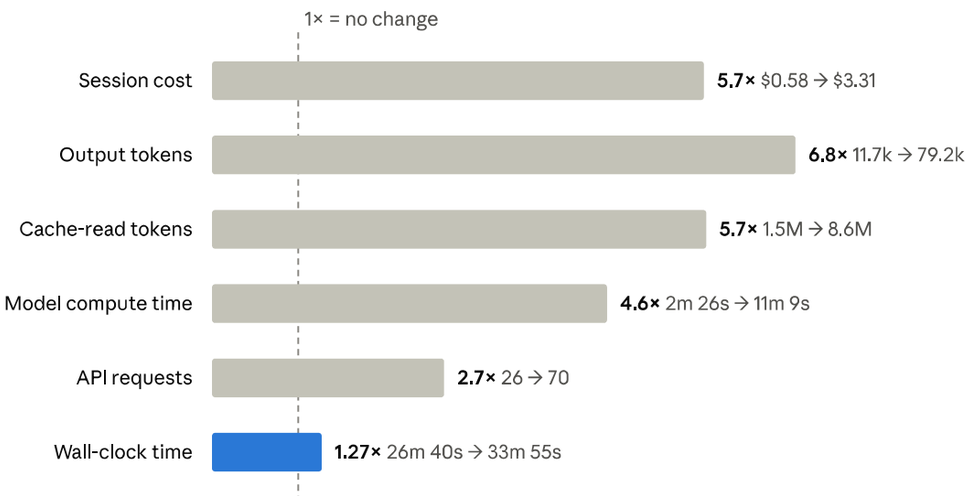
What was measured: session cost, tokens and time (from Claude Code's session summary); source, test and document line counts; a code review of both repos; the Groundwork test suite and `groundwork check`, run again for this report; and a page-by-page review of both 21-page PDFs made from the same test document, a public Markdown feature cheat sheet.

Cost, speed and token efficiency

Groundwork cost $2.73 more and used 8.7 more minutes of model compute, but added only 7 minutes 15 seconds of wall-clock time.

Model compute rose 4.6–6.8×; wall-clock time rose only 1.27×

With Groundwork ÷ without, same model (Sonnet 5.5). Labels show without → with.



- **The extra spend bought extra work, not waste.** Both sessions served 98% of input tokens from cache with no misses. The loop was equally efficient; Groundwork simply produced more: about 1,900 lines on disk (675 app, 680 test, 539 document) against 159.
- **Per line delivered, Groundwork was cheaper.** It spent about 42 output tokens per line on disk, against about 74 without. The ad-hoc session's "0 lines added" is a counter artefact: it wrote files through the shell, which the counter does not track.
- **Most of the wall-clock time is human time.** The ad-hoc session used 2.4 minutes of compute in 26.7 minutes of wall-clock. In the Groundwork run, the RFC was opened at 04:24 and tested code was done at 04:42 UTC: 18 minutes including the interview and two sign-offs.

- **Unit economics:** $3.31 bought 36 automated tests (about $0.09 each) and 15 traced requirements. The $0.58 build has nothing to measure it against.

Documentation and traceability

The Groundwork repo can explain itself; the ad-hoc repo cannot. Groundwork produced 539 lines of documents across 11 files, every one cross-linked by requirement ID. The ad-hoc build produced none, not even a README.

Document	What it answers	Without	With
`PROJECT.md`	Why this exists, who owns it, what is out of scope	—	Yes
`CONSTITUTION.md`	Rules every change is checked against, with dated amendments	—	Yes
`ARCHITECTURE.md`	Components, data flow, accepted risks	—	Yes
`AGENTS.md`	How any coding agent must work in this repo	—	Yes
`README.md`	How to run, use and stay safe	—	Yes
`RFC-0001`	The decision, the interview record, alternatives rejected, risks accepted, who approved	—	Yes
`spec.md`	What to build: 11 FRs, 4 NFRs, 12 acceptance criteria, failure behaviour, manual test	—	Yes
`plan.md`	How to build it, each choice citing an FR	—	Yes
`tasks.md`	11 reviewable tasks plus 4 verification steps, each with "done when"	—	Yes
`evals.md`	24 test scenarios written before the code, plus edge cases nobody asked about	—	Yes
`log.md`	Where work stopped and what is next	—	Yes

Why this matters:

- **Every line of code traces to a reason.** Code comments cite requirement IDs (`FR-11` , `NFR-2`); tests are named after acceptance criteria (`AC-7`); the plan cites the spec; the spec cites the RFC.
- **Approvals are tamper-evident.** `.groundwork/approvals.json` stores a hash of each approved document with the signer and time. An edit after approval is detectable.
- **Docs stay current.** `groundwork check` reports "conforms to standard 0.5.0 — no findings" and `groundwork fresh` reports all four foundation documents FRESH (re-run for this report).
- **The spec caught a gap before code.** While writing evals, the agent found the spec never said what happens on an empty editor. It asked the owner, recorded the answer and added AC-12.

Testing and verification

The Groundwork build ships 36 automated tests that all pass in 23 seconds (re-run for this report). The ad-hoc build ships zero, and its `npm test` script exits with an error.

Test file	Tests	What it proves
`ui.test.js`	13	Live preview, upload, wrong file type, file names, empty and oversized input, saved state, keyboard access, narrow layout, double-click
`pdf.test.js`	10	Page size, orientation, margins, page numbers, dark theme, warnings, invalid input, browser-crash recovery, 3 concurrent exports
`render.test.js`	8	Tables, task lists, code colour, math, diagrams, bad diagram, bad math, images, themes, hostile script, Unicode
`guards.test.js`	4	Bound to 127.0.0.1, foreign host refused, 1 MB limit both sides of the boundary
`nowrite.test.js`	1	Exporting writes no files and logs no content

- **Tests were designed before the code.** `evals.md` listed 24 scenarios, each tied to an acceptance criterion, plus a catalogue of things real users do that nobody wrote down.
- **Every acceptance criterion has a test**, which is a Constitution quality gate.
- **The gaps are named, not hidden.** The task list marks T902, the hand check of PDF against preview, as not done. That unchecked box is exactly where this review found the layout defects.

Code quality and architecture

The Groundwork code is structured, testable and safe by default for its network exposure. The ad-hoc code is compact and readable but untested, exposed to the network, and its preview can drift from the PDF.

Dimension	Without Groundwork	With Groundwork
Size	159 lines in 3 files	675 app lines in 12 files
Structure	One `server.js` does everything; UI and script inline in one HTML file	Server, PDF step, shared renderer, UI, themes in separate modules; `createApp()` factory allows testing
Preview matches PDF	No: preview and PDF use different stylesheets	Yes: one `render.js` and one set of theme CSS feed both, a Constitution rule
Network exposure	Listens on all interfaces, no host check: anyone on the same Wi-Fi can use it	Binds to 127.0.0.1 and rejects foreign `Host` headers (blocks DNS rebinding)
Untrusted content	Raw HTML off, JavaScript off in the PDF browser: safer	HTML and scripts on (needed for math and diagrams); risk written down and signed off

Input validation	Page size only; everything else passed through	Every field allow-listed; clear 400 / 413 messages that never echo content
Failure handling	Generic 500; browser auto-restarts	Broken images and diagrams become placeholders plus warnings; browser crash recovers; tested
Live preview	Server round trip every 250 ms of typing	Renders in the browser, no server load
Features	Paste, upload, live preview, A4/Letter	Plus 4 themes, orientation, margins, page numbers, A3/Legal, LaTeX math, Mermaid diagrams, saved state, smart file names
Hygiene	`npm test` exits with an error; `main` points to a missing file	`npm start` / `npm test` work; lockfile committed; README run steps verified

Defects found in the Groundwork code during this review:

- Long code lines are cut off at the right edge of the PDF. The stylesheet uses horizontal scrolling, which paper cannot do.
- Headings are not kept with the text that follows, so five headings sit alone at the bottom of a page.
- The evals file cites `test/themes.test.js`, which does not exist. The theme checks live in two other test files.

None of these is caught by the 36 tests, because no test inspects visual layout. That check is the spec's manual test, which the task list still marks as not done.

Output quality: the two PDFs

Both apps turned the same 21-page cheat sheet into a readable A4 PDF. Groundwork's PDF supports more of the document and labels its failures; the ad-hoc PDF paginates and wraps more cleanly.

Element	Without Groundwork	With Groundwork	Better
Headings, emphasis, lists, blockquotes, rules	Correct	Correct	Tie
Code highlighting	Correct, GitHub colours	Correct, theme colours	Tie
Tables	Correct, full width	Correct, sized to content	Tie
Web images	All 5 load	All 5 load	Tie
Long code lines	Wrap onto the next line	Cut off at the right edge in at least 6 blocks	Without

Page breaks	Headings stay with their text	5 headings stranded at the bottom of a page (Lists, Task lists, Links, Footnotes, Tables)	Without
Inline HTML (`<dl>` , `<del>`)	Printed as raw tags	Rendered as a definition list and strikethrough	With
Embedded YouTube thumbnail	Raw HTML printed as text	Rendered as an image	With
Broken image	Broken-image icon and alt text	Labelled red placeholder, plus a warning in the app	With
Page numbers	None	"n / 21" footer on every page	With
Theme	One fixed look	Clean document (serif) chosen; 3 more available	With
Footnotes	Not supported (printed as `[^first]`)	Not supported	Tie: neither asked

Reading the result: the two defects in the Groundwork PDF are cosmetic CSS fixes of a few lines each. The ad-hoc gaps (no themes, no page setup, no math or diagrams, raw HTML) are missing features. Neither build supports footnotes, because neither the prompt nor the Groundwork interview raised them; a spec makes such omissions visible for review, an ad-hoc build does not.

What this means at production scale

On a one-person toy app, Groundwork's benefits are nice to have. On a multi-team, user-facing product, they address the failures that cost the most. The table below projects each finding from this test onto our production setting; these are reasoned expectations, not measured results.

Production problem	What happened in this test	What Groundwork adds at scale
Vague tickets become wrong features	A one-sentence prompt; the ad-hoc agent guessed every requirement	A mandatory interview and an approved spec before code; product owners sign what will be built
Risky trade-offs made silently	Ad-hoc build exposed the server to the network without anyone deciding	Tensions stop the agent and go to a human; the decision, risk and signer are recorded in an RFC
Agents and engineers drift apart	—	Every agent reads the same `AGENTS.md` , Constitution and architecture vocabulary, so 10 agents behave like one team
Cross-team breakage	Single repo, not exercised	RFCs classed `api` need sign-off from each affected lead and record the cross-repo contract

Review bottleneck on AI-written code	159 untested lines vs 1,355 lines of code and tests	Reviewers check code against a numbered spec and named tests instead of reverse-engineering intent
Slow onboarding and handover	Ad-hoc repo has no README	New people and agents read PROJECT, ARCHITECTURE, spec and log; `log.md` says where work stopped
Audit and compliance questions	—	Hash-stamped approvals, a ledger of document events, and dated Constitution amendments
Standards decay over time	—	Model-free `groundwork check` runs locally, in git hooks and in CI, so conformance does not depend on anyone remembering

The cost argument at scale: the premium in this test was $2.73 and about 9 minutes of compute. On a production feature, the comparable cost of skipping the process is a mis-built feature, a security review finding, a production incident or a week of onboarding. Each of those costs engineer-days, not dollars. The extra model spend is the cheapest line in that ledger.

Limitations and next steps

This is one run per condition on a small app, so it shows direction, not statistical proof.

Limitations of this test:

- **Sample of one.** Model output varies between runs; repeat each condition 3–5 times before quoting ratios externally.
- **Unequal scope.** Groundwork's interview grew the scope (themes, math, diagrams), so part of the cost premium is extra features, not process overhead.
- **Enforcement not exercised.** The repo had no commits, so git hooks and CI checks never ran. Multi-repo RFCs and multi-signer approvals were also not tested.
- **Wall-clock time is noisy.** It includes human reading and idle time in both sessions.
- **Groundwork accepted a real risk.** The owner chose to let pasted Markdown run scripts in the PDF browser. That is fine for a local tool and documented, but the same choice on a hosted product would need a security review; the RFC gate is where that review would happen.

Recommended next steps:

- [ ] Repeat this benchmark 3 times per condition, plus one larger multi-feature app, and report the spread.
- [ ] Run a one-quarter pilot on one production feature team, with Groundwork's `check` in their git hooks and CI.
- [ ] Track pilot metrics against a comparable team: escaped defects, PR review time, rework after review, onboarding time for a new member.
- [ ] Add an automated visual check (render PDF pages and compare to preview) so layout defects like those found here are caught without the manual step.

- [ ] Decide which approval roles map to our org: product owner for RFCs, tech lead for specs, each affected lead for cross-repo `api` RFCs.